

外卖配送任务分配 AutoSolver Agent 技术报告

冯文璐
张 硕

fengwenlu25@mails.ucas.ac.cn
zhangs21@mails.tsinghua.edu.cn

一、任务背景与设计目标

外卖配送任务分配问题的核心是在接单概率、配送分数和合单机会之间做稳定权衡。本项目的输入是已经预计算好的任务组、骑手、`total_score` 和 `willingness` 组合；系统需要从这些候选组合中选择若干行输出，使订单尽可能被接起，同时让期望惩罚分数尽可能低。由于一个订单可以同时指派给多位骑手，合单任务也可能覆盖多个订单，简单的一对一匹配无法完整表达题目结构。

本项目为一个自动研究型求解器生成问题，更关注系统是否能自主尝试不同策略、是否能在每次实验后自动评估并保留更优结果。项目设计的重点放在 AutoSolver Agent 闭环上：由 Agent 读取算例特征和算法组件，调用 Deepseek-v4-pro 大模型生成候选 `solver.py`，再用本地评测结果反向约束下一轮生成方向。最终交付是一个满足在线评测平台接口的 `solver.py` 文件。`solve(input_text: str)` 函数内部不依赖外部网络、文件系统或第三方包，需要在 10 秒限制内返回`[(task id list,[courier id]),...]`形式的方案。

二、系统总体设计

系统采用“离线自动研究、在线轻量求解”的两层结构。离线阶段由 `autosolver_agent.py` 负责组织多轮实验，在线提交阶段只保留生成出的 `best_generated.py` 或 `final_generated.py` 中的 `solve` 函数。这样的分层能让 LLM 探索、代码生成和多轮评估发生在赛前，同时保证提交文件运行时没有额外依赖。

算法知识以组件库形式提供给 Agent，而不是把完整最终答案直接固定下来。`algorithm_components` 目录下包含输入解析、目标函数、初始构造、匹配骨架、局部搜索和搜索调度等模块，Agent 会按阶段把这些材料节选进提示词，让 LLM 在可控范围内选择、组合和改写策略。这个设计避免了每轮只做随机改参数，也让每次候选代码都能表达一个明确的算法假设。

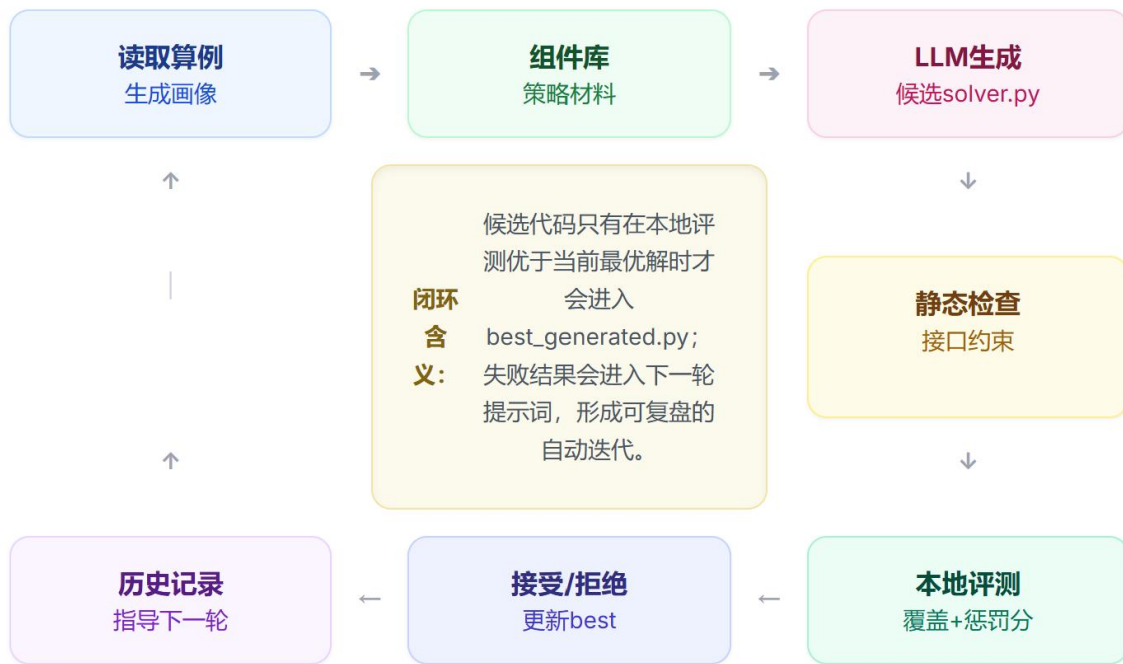


图 1 AutoSolver Agent 自动研究闭环。

系统支持单算例和多算例两种运行方式。单算例运行适合针对某个目标 case 做深挖，让提示词和搜索预算集中服务于一个算例；多算例运行适合检验求解器在不同规模、不同骑手密度和不同接单意愿场景下的稳定性，Agent 会用聚合覆盖数和聚合惩罚分筛选候选代码。

本项目测试时所采用的 large_seed301 是赛事官方真实算例，其余 9 个 case 为围绕不同场景生成的虚拟算例。虚拟算例包括 tiny、small、medium、large、low_willingness、scarce_couriers 和 high_noise 等类型，作用是让 Agent 在公开真实 case 之外接受泛化压力，避免策略只贴合单一输入。



图 2 真实算例与虚拟算例的实验角色。

三、自主策略探索机制

自主探索不是让模型随意生成代码，而是让模型在受约束的策略空间中提出可验证实验。Agent 会把算例画像写入提示词，例如 large_seed301 包含 40 个任务、80 个骑手、820 个任务组、33780 个候选组合，且具有 large、rider_rich、pair_heavy 等标签；这些特征会引导模型优先考虑合单、骑手富余和局部换骑手策略。

组件释放采用课程式和自适应结合的方式。迭代开始时先考虑基础解析、目标函数、贪心构造、合单种子和少量局部搜索，目标是在迭代前期用较少的轮数得到第一个有效 solver；当系统已经覆盖全部任务后，后续轮次会把重点切换到降低惩罚分数，并采用 repair_window、bundle_plan_seed、cycle_riders、path_riders、config_polish、LNS 和全局 orchestration 等更强组件进行深度优化。

单算例模式下，Agent 的优化焦点更集中。以 large_seed301 为例，界面只选择该真实 case 时，系统会围绕 40 个任务和 80 个骑手的 rider_rich、pair_heavy 结构持续调整搜索方向。多算例模式下，Agent 的探索目标转为整体组合更稳。多算例运行同时包含 1 个真实算例和 9 个生成虚拟算例，候选 solver 必须在聚合指标上超过当前最优才会被接受，因此更能体现自动研究过程的泛化约束。

四、候选求解器评估与筛选

自动评估使用统一的代理目标函数，避免依赖人工主观判断。对每个任务组和骑手集合，系统根据 willingness 计算至少一名骑手接单的概率，同时用 total_score 估计接起后的期望成本；若任务未被接起，则按每个任务 100 分加入拒单惩罚。整体比较规则是先最大化覆盖任务数，再最小化惩罚分数。



图 3 代理目标函数把接单概率、score 和拒单惩罚统一到同一评估分数中。

候选代码进入评测前会先经过静态保护。检查内容包括必须定义 `solve(input_text)`，不能超过大小限制，不能使用不允许的导入，不能依赖网络、子进程或硬编码算例名称，并保持 Python 3.6 兼容。这个环节可以提前过滤明显不符合提交要求的生成结果，降低后续评测成本。

评测通过后，Agent 只在候选结果优于当前最优解时更新 `best_generated.py`。对于单算例运行，比较对象是该 case 的覆盖数和惩罚分；对于多算例运行，比较对象是全部 case 的聚合覆盖数和聚合惩罚分。这个统一规则让系统既能做专项优化，也能做跨场景筛选。

五、UI 界面与使用流程

项目提供 `dashboard.py` 作为可视化操作入口，用于把命令行 Agent 流程封装成更容易复现实验的界面。用户通过 `start_dashboard.bat` 启动 Streamlit 界面后，可以在页面中选择测试算例、模型预设、生成轮数、单轮超时时间、提示词模式、组件字符预算和 `best` 代码回填长度等参数。



图 5UI 初始界面：选择 case、设置迭代轮数后，点击“启动 Agent”即可开始生成 `solver.py`。

UI 的核心使用流程是“选择算例—配置实验—启动运行—观察结果—导出最优解”。在算例选择区只勾选 `large_seed301.txt` 时，系统进入单算例训练/评测模式；同时勾选多个 case 时，系统进入多算例模式，界面会提示当前选择数量，并在上传网站前支持用部分 case 进行训练和预测。

运行过程中，UI 会持续读取 agent.log、history.jsonl 和 summary.md 展示实验进展。用户可以看到每一轮候选的状态、是否被接受、覆盖任务数、惩罚分数、运行时间和拒绝原因；当出现多轮结果比较时，界面也会展示 refine 和 recompose 的胜出情况。这个界面让迭代过程不再只是日志文件，而是可观察、可复盘的实验记录。

UI 还承担了最终文件管理功能。每次运行的最优候选会保存为 best_generated.py，候选源码和大模型原始回复会保存在 candidates 目录下；当用户确认某次运行效果最好时，可以把 best_generated.py 应用到提交路径，得到最终 solver.py 或 final_generated.py。

六、运行结果与迭代分析

单算例运行验证了 Agent 可以把搜索集中到一个真实目标 case 上。截图中只选择 large_seed301.txt，系统完成 6 轮迭代，最佳覆盖数为 40，最佳惩罚分为 652.769，Case 表现诊断显示完成算例 1/1。

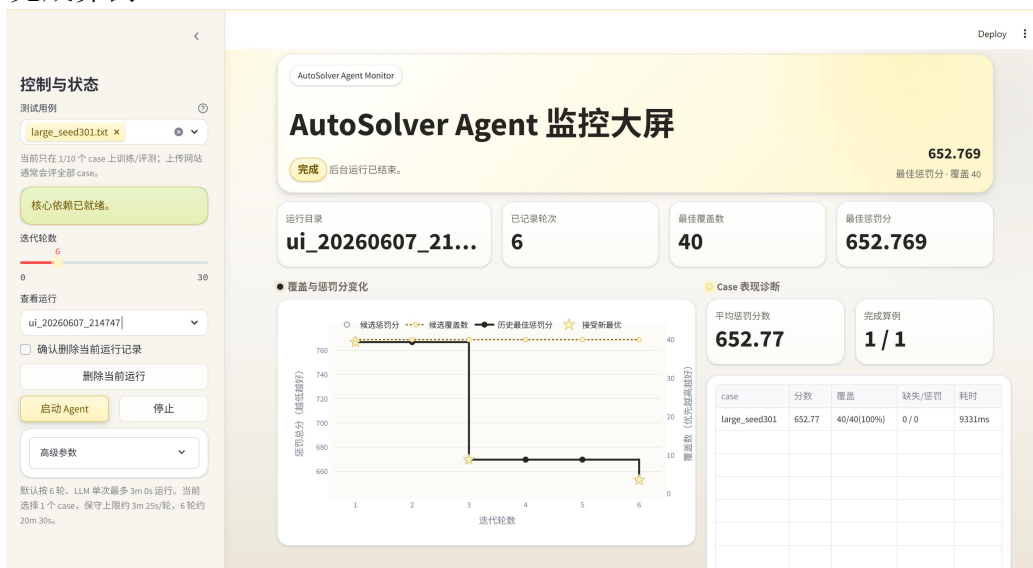


图 6 单算例运行界面：large_seed301 在 6 轮迭代后达到覆盖 40、最佳惩罚分 652.769。

多算例运行验证了 Agent 可以在更复杂的组合场景中同时优化多个 case。截图中选择 10 个测试用例，其中 large_seed301 为真实算例，其余 9 个为生成的虚拟算例，最终最佳覆盖数为 291，最佳惩罚分为 5944.641，完成算例 10/10。

外卖配送任务分配 AutoSolver Agent 技术报告

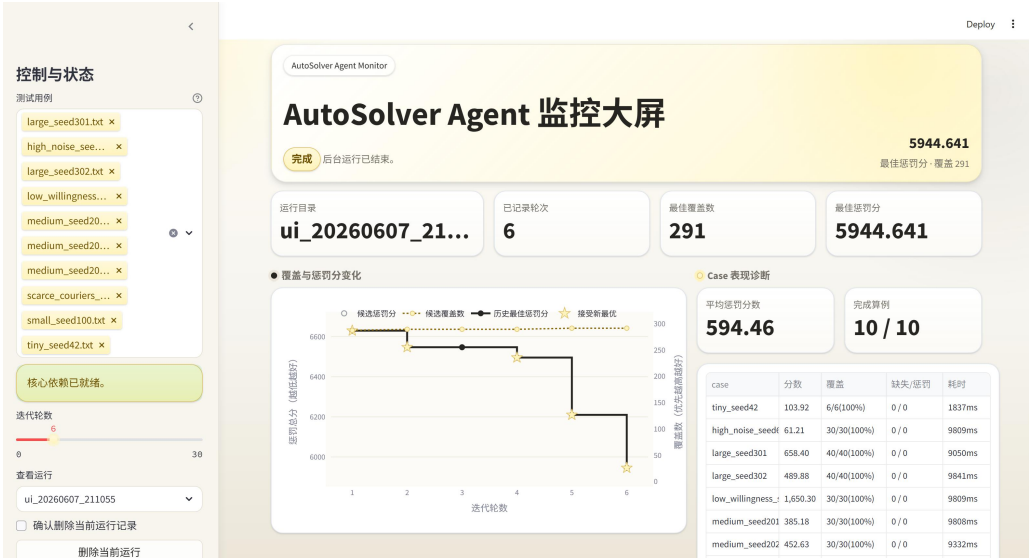


图 7 多算例运行界面：1 个真实 case 和 9 个虚拟 case 共同评测，最终聚合覆盖 291、聚合惩罚分 5944.641。

模式	算例来源	选择 case	迭代轮数	最佳覆盖	最佳惩罚分	完成情况
单算例	真实算例	large_seed301	6	40	652.769	1/1
多算例	1 真+9 虚拟	10 个 case	6	291	5944.641	10/10

多算例 summary 进一步说明了多轮自动迭代的收益。第 1 轮得到首个有效 solver，聚合覆盖 287、惩罚分 6630.350017；第 2 轮提升到覆盖 289、惩罚分 6547.527203；第 3 轮结果重复而被拒绝；第 4 轮通过 refine 降到 6496.030467；第 5 轮覆盖提升到 291 且惩罚分降到 6209.573021；第 6 轮 recompose 继续降到 5944.641255 并被接受。

轮次	状态	胜出变体	聚合覆盖	聚合惩罚分	决策原因
1	accepted	candidate	287	6630.350017	首个有效生成 solver
2	accepted	candidate	289	6547.527203	聚合指标改进
3	rejected	candidate	289	6547.527203	未优于当前最优
4	accepted	refine	289	6496.030467	惩罚分下降
5	accepted	candidate	291	6209.573021	覆盖和惩罚分改进
6	accepted	recompose	291	5944.641255	聚合惩罚分继续下降

最终多算例候选在 10 个 case 上均达到 100%覆盖。

单算例和多算例结果之间也体现了两种运行方式的取舍。单算例 large_seed301 可以把提示词和时间预算集中在一个真实 case 上，结果为 652.769；多算例最终 solver 在 large_seed301 上的分数为 658.397，略高于单算例专项结果，但它同时保证 1 个真实算例和 9 个虚拟算例全部完成，并把聚合惩罚分从 6630.350017 降到 5944.641255。也就是说，系统既能针对单一情景案例提供解决算法，也能在多场景下自动寻找更稳的综合解。

七、在线评测平台验证

在线评测平台结果是对本地 AutoSolver Agent 流程的最终验收。本地阶段得到的 best_generated.py 会被整理为提交用 solver.py，并上传到官方提供的在线评测系统；在线系统独立运行全部 case 后返回平均惩罚分、完成算例数、各 case 覆盖情况和耗时，因此可以验证本地评估闭环是否真正转化为可提交结果。

本次上传 Agent 提供的代码后，官方平台显示评测完成，平均惩罚分数为 701.97，完成算例为 10/10。各 case 均达到 100%覆盖，其中真实算例 large_seed301 在线分数为 650.13，覆盖 40/40；其余 9 个算例也全部完成，包括 high_noise_seed601、large_seed302、low_willingness_seed501、medium_seed201、medium_seed202、medium_seed203、scarce_couriers_seed401、small_seed100 和 tiny_seed42。

该结果说明，Agent 生成的最终 solver 不仅能在本地 UI 中完成单算例和多算例评测，也能通过官方在线平台的独立运行环境。在线结果与本地结果在绝对分值上存在差异，主要来自评测平台使用官方环境、官方汇总方式和最终提交代码版本；但二者都体现了同一个结论：系统能够在 10 个 case 上保持完整覆盖，并把自动迭代筛选出的代码转化为可提交方案。

✅ 评测完成			
平均惩罚分数	701.97		
完成算例	10 / 10		
high_noise_seed601	474.92	30/30(100%)	9764ms
large_seed301	650.13	40/40(100%)	9748ms
large_seed302	623.66	40/40(100%)	9779ms
low_willingness_seed501	1,799.90	30/30(100%)	9752ms
medium_seed201	472.83	30/30(100%)	9745ms
medium_seed202	518.72	30/30(100%)	9756ms
medium_seed203	492.52	30/30(100%)	9759ms
scarce_couriers_seed401	1,542.09	40/40(100%)	9743ms
small_seed100	293.28	15/15(100%)	4784ms
tiny_seed42	151.64	6/6(100%)	1768ms

图 9 官方在线评测平台最优结果：平均惩罚分 701.97，完成算例 10/10。

八、总结

本项目完成了一个面向配送任务分配的 AutoSolver Agent 系统。系统不是人工指定单一算法，而是让大模型在组件库和算例画像指导下提出候选策略，再由本地评测器自动筛选，形成生成、评估、记录和改进的闭环。离线 Agent 阶段可以调用 DeepSeek-v4-pro、查看历史实验、组合贪心、合单、局部搜索和全局重组等策略；在线 solver 阶段则保留轻量、独立、可限时运行的组合启发式。单算例运行说明系统能围绕真实 case 持续压分，多算例运行说明系统能在多个实际情景中保持完整覆盖并优化聚合惩罚分。

从工程角度看，UI 界面增强了系统可用性和可复现性。用户可以通过页面配置算例、模型、轮次和预算，实时查看每轮候选的接受或拒绝原因，并导出当前最优解。这样的设计让 AutoSolver 不只是一个最终 solver.py 文件，也是一套能解释自己如何变好的自动研究流程。